Activity 1: Proof of Concept!

Direction: Follow the direction and write the code accordingly.

Part 1: Asymmetric Cipher Handler

1. In your folder directory, create **cipher.py** and import the following modules:

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
```

2. Open new terminal, download and install cryptography module:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS D:\Lecture\AY-2024-2025-First-Sem\ASA\Simple End to End Encryption Chat Program> pip install cryptography
```

3. Create a function that generates public and private key:

```
#Generates public key and private key
def generatePrivateAndPublicKey():
    #Generate private key
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

    #Get public key
    public_key = private_key.public_key()

    #Serialize public key
    pem_public = public_key.public_bytes(encoding=serialization.Encoding.PEM,
                                         format=serialization.PublicFormat.SubjectPublicKeyInfo)
    #Serialize private key
    pem_private = private_key.private_bytes(encoding=serialization.Encoding.PEM,
                                           format=serialization.PrivateFormat.TraditionalOpenSSL,
                                           encryption_algorithm=serialization.NoEncryption())

    #Save public key
    with open('public_key.pem', 'wb') as f:
        f.write(pem_public)

    #Save private key
    with open('private_key.pem', 'wb') as f:
        f.write(pem_private)

    return pem_public
```

4. Create a function that load your peer public key:

```python
#Load Receiver Public Key
def loadReceiverPublicKey(message):
    receiver_public_key = serialization.load_pem_public_key(message)
    return receiver_public_key
```

5. Create a function that encrypts message:

```python
#Encrypt Message using the reciever's public key
def encryptMessage(message, public_key):

    encryptedMessage = public_key.encrypt(
        message,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    return encryptedMessage
```

6. Create a function that decrypts message:

```python
#Decrypt Message using private key
def decryptMessage(encryptedMessage):

    # Load the private key to decrypt the message
    with open('private_key.pem', 'rb') as f:
        private_key = serialization.load_pem_private_key(f.read(), password=None)

    decryptedMessage = private_key.decrypt(
        encryptedMessage,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    return decryptedMessage
```

Part 2: Socket Communication Handler

1. In your folder directory, create **connect.py** file and import the following modules:

```
import threading
import socket
from cipher import generatePrivateAndPublicKey, loadReceiverPublicKey, encryptMessage, decryptMessage
```

2. Initialize your public and private key and your peer's public key:

```
#Generate Public and Private Key
my_public_key = generatePrivateAndPublicKey()

#Initialize global variable for peer's public key
peer_public_key = None
```

3. Create a function that send's message to other pc:

```
def send_message(conn):
    global peer_public_key

    #Send public key first
    try:
        conn.sendall(my_public_key)
        print("Public key sent")
    except (ConnectionResetError, OSError) as e:
        print(f"Error sending public key: {e}")
        return

    while True:
        try:
            message = input("You: ").encode('utf-8')

            #Encrypt message using peer's public key
            if peer_public_key:
                encrypted_message = encryptMessage(message, peer_public_key)
                conn.sendall(encrypted_message)
            else:
                print("Waiting for peer's public key....")
        except (ConnectionResetError, OSError) as e:
            print(f"Error sending message: {e}")
            break
```

4. Create a function that receive message from other pc:

```
def receive_message(conn):
    global peer_public_key

    while True:
        try:
            if peer_public_key is None:
                peer_public_key_data = conn.recv(2048)

                if peer_public_key_data:
                    peer_public_key = loadReceiverPublicKey(peer_public_key_data)
                    print("Peer's public key received!")
                else:
                    print("Failed to receive peer's public key.")
                    break

            else:
                encryptedMessage = conn.recv(2048)
                if not encryptedMessage:
                    print("Connection closed by peer.")
                    break

                #deccrypt message
                decryptedMessage = decryptMessage(encryptedMessage)
                print(f"Peer: {decryptedMessage.decode('utf-8')}")

        except (ConnectionResetError, OSError) as e:
            print(f"Error receiving message: {e}")
            break
```

5. Create a function that start client mode:

```
def start_client_mode(server_ip, port):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        try:
            print(f"Attempting to connect to {server_ip}:{port}")
            s.connect((server_ip, port))
            print(f"Connected to {server_ip}:{port}")


            #Start thread to send and receive messages
            receive_thread = threading.Thread(target=receive_message, args=(s,))
            send_thread = threading.Thread(target=send_message, args=(s,))

            receive_thread.start()
            send_thread.start()

            #Keep both thread running
            receive_thread.join()
            send_thread.join()

        except (ConnectionRefusedError, OSError) as e:
            print(f"Failed to connect to server: {e}")
```

6. Create a function start server mode:

```
def start_server_mode(host, port):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((host,port))
        s.listen()
        print(f"Waiting for a connection on {host}:{port}")
        conn, addr = s.accept()
        print(f"Connected by {addr}")


        #Start thread to send and receive messages
        receive_thread = threading.Thread(target=receive_message, args=(conn,))
        send_thread = threading.Thread(target=send_message, args=(conn,))

        receive_thread.start()
        send_thread.start()

        #Keep both thread running
        receive_thread.join()
        send_thread.join()
```

Part3: Main App

1. In same folder directory, create **app.py** file and import the following module:

```
from connect import start_client_mode, start_server_mode
```

2. Build the app. Under **host** change it to your ip address:

```
if __name__ == "__main__":
    mode = input("Enter mode (server/client): ").strip().lower()

    if mode == 'server':
        host = '0.0.0.0' #Use local ip address
        port = 65432
        start_server_mode(host, port)

    elif mode == 'client':
        server_ip = input("Enter server IP address: ").strip()
        port = 65432
        start_client_mode(server_ip, port)

    else:
        print("Invalid mode! Please enter server or client")
```

Part 4: Test and Check

In order for the app to work you need to find a partner. One of you must be the server and the other is the client. To make sure the app work, the **server** must be able to send message to **client** and **vice versa**.